🔐 Java Full-Stack JWT Role Handshake

Spring Boot + Angular Role-Based Access Control (RBAC) — Beginner ➜ Production Guide

Mental Model:
JWT works like a **digital passport**.

- 🏛️ **Spring Boot** → Embassy issuing the passport
- 🎫 **JWT Payload** → Visa stamps (roles)
- 🛂 **Angular** → Border control checking permissions

This guide shows the **complete professional flow** used in enterprise Java full-stack systems.

🧭 Architecture Overview

User Login
↓
Spring Boot Authentication
↓
JWT Created (with roles claim)
↓
Angular stores token
↓
Angular decodes token → reads roles
↓
Route Guards protect UI
↓
Spring Boot verifies roles again

Important Rule

✔ Frontend guards → User Experience
✔ Backend authorization → Real security

🏗️ STEP 1 — Spring Boot: Stamp Roles in JWT

When a user logs in, Spring Boot generates a JWT containing user roles.

Example JWT Payload

```
{
  "sub": "shashank",
  "roles": ["ROLE_ADMIN","ROLE_USER"],
  "iat": 1710000000,
  "exp": 1710086400
}
```

The important field is:

roles

Angular will read this.

⚙️ Spring Boot Implementation

Token Provider

```
public String generateToken(Authentication authentication) {

    UserDetails userPrincipal =
        (UserDetails) authentication.getPrincipal();

    List<String> roles = userPrincipal.getAuthorities()
        .stream()
        .map(GrantedAuthority::getAuthority)
        .collect(Collectors.toList());

    return Jwts.builder()
            .setSubject(userPrincipal.getUsername())
            .claim("roles", roles) // Custom Claim
            .setIssuedAt(new Date())
```

```
        .setExpiration(
            new Date(System.currentTimeMillis() + 86400000)
        )
        .signWith(SignatureAlgorithm.HS512, secretKey)
        .compact();
}
```

Key Line

.claim("roles", roles)

This embeds roles into the token.

📦 Example JWT Token

HEADER.PAYLOAD.SIGNATURE

Decoded payload:


```
{
 "sub":"admin",
 "roles":["ROLE_ADMIN","ROLE_USER"],
 "exp":1710000000
}
```


Angular only reads the **payload**.

⚡ STEP 2 — Angular: Decode JWT

JWT is Base64 encoded.

Angular can decode it **without secret key**.

Install library:

npm install jwt-decode

🧠 AuthService Implementation

```
import jwt_decode from 'jwt-decode';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {

  getRoles(): string[] {

    const token = localStorage.getItem('token');

    if (!token) return [];

    const decoded: any = jwt_decode(token);

    return decoded.roles || [];
  }

  hasRole(expectedRole: string): boolean {
    return this.getRoles().includes(expectedRole);
  }
}
```

🔎 Example Usage

```
if (this.authService.hasRole('ROLE_ADMIN')) {
   console.log("Admin access granted");
}
```

🛡️ STEP 3 — Angular RoleGuard

Protect routes based on roles.

RoleGuard Implementation

```
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {

  constructor(
    private authService: AuthService,
    private router: Router
  ) {}

  canActivate(route: ActivatedRouteSnapshot): boolean {

    const expectedRole = route.data['role'];

    if (this.authService.hasRole(expectedRole)) {
      return true;
    }

    this.router.navigate(['/unauthorized']);
    return false;
  }
}
```

🗺️ STEP 4 — Route Configuration

```
const routes: Routes = [

  {
    path: 'admin-panel',
    component: AdminComponent,
    canActivate: [RoleGuard],
    data: { role: 'ROLE_ADMIN' }
  }

];
```

👨‍💻 Example Scenario

User Roles:

ROLE_USER

Route requires:

ROLE_ADMIN

Result:

Access Denied
Redirect → /unauthorized

🚀 STEP 5 — Protect UI Elements

Hide UI elements based on role.

```
<button *ngIf="authService.hasRole('ROLE_ADMIN')">
  Delete User
</button>
```

🌍 STEP 6 — Angular HTTP Interceptor (Auto Attach JWT)

Instead of adding token manually everywhere.

Create interceptor.

JWT Interceptor

```
@Injectable()
export class JwtInterceptor implements HttpInterceptor {

  intercept(
    request: HttpRequest<any>,
    next: HttpHandler
  ): Observable<HttpEvent<any>> {

    const token = localStorage.getItem('token');

    if (token) {

      request = request.clone({
        setHeaders: {
          Authorization: `Bearer ${token}`
        }
      });

    }

    return next.handle(request);
  }
}
```

Register Interceptor

```
providers: [
{
 provide: HTTP_INTERCEPTORS,
 useClass: JwtInterceptor,
 multi: true
}
]
```

Now every request automatically includes:

Authorization: Bearer <JWT>

🔒 STEP 7 — Backend Security (Most Important)

Angular security **can be bypassed**.

Backend must enforce security.

Spring Security Method Protection

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/users")
public List<User> getUsers() {
    return userService.getAllUsers();
}
```

Enable Method Security

```
@EnableMethodSecurity
@Configuration
public class SecurityConfig {
}
```

🏗️ Full Enterprise Flow

Login
↓
Spring Boot Authentication
↓
JWT Generated (roles embedded)
↓
Angular stores token
↓
Angular HTTP interceptor sends token
↓
Angular guards protect routes

↓
Spring Security verifies roles again
↓
API response returned

📋 Enterprise Checklist

Layer	Responsibility
Spring Boot	Generate JWT with roles
Angular AuthService	Decode token
Angular RoleGuard	Protect routes
Angular Interceptor	Attach JWT
Spring Security	Validate roles

⚠️ Common Mistakes

❌ Trusting frontend roles
❌ Storing sensitive data in JWT
❌ Forgetting token expiration handling
❌ Missing backend role verification

🧠 Pro Tip

Never trust Angular alone.

Always verify roles again in Spring Boot.

Frontend Security = UX
Backend Security = Real Security

🏆 Final Wisdom

A beginner builds authentication.

A professional builds **authorization architecture**.

A master builds **defense-in-depth security**.

JWT roles + Angular guards + Spring Security
 = secure full-stack RBAC system.

Next steps:

1️⃣ **Complete Spring Boot + Angular JWT project structure**
2️⃣ **Refresh token architecture (enterprise standard)**
3️⃣ **JWT logout & token blacklisting strategy**
4️⃣ **Production-grade security architecture diagram**